

CAN Fuoco: Спецификация протокола CAN для общения с драйверами моторов робота FootBot4

Содержание

Введение	2
Об интерфейсе CAN	2
Канальный уровень	3
Транспортный уровень	5
Уровень приложения	5
О протоколе ISO-TP	5
Краткая спецификация протокола	5
О протоколах уровня приложения	6
Протокол CanOpen	6
Протокол XCP	6
Формат файлов .a2l	6
Резюме	6
Протокол UDS	7
Постановка задачи	8
Задачи материнской платы	8
Задачи драйвера	8
Технические ограничения	8
Характеристики МК материнской платы	8
Характеристики МК драйвера	8
Общие требования	8
Определение частоты цикла системы управления	8
Спецификация протокола	9
Плюсы подхода	9
Формат заголовка	9
affected_id	10
register_id	10
Полезная нагрузка	10
Таблица регистров	10
Описание виртуальных регистров	15
Регистр MULTI_TARGET	15
Регистр SHAFT_FEEDBACK	15
Спецификация FP16	15
Библиография	17

Введение

Программное обеспечение материнской платы и драйверов новых роботов FootBot4 разрабатывается нашей командой по сути с нуля, в связи с чем существует необходимость установить протокол обмена информацией между материнской платой и драйверами для последующего его реализации в ПО.

Для связи между материнской платой и драйверами используется интерфейс CAN. Данный документ описывает протокол обмена для взаимодействия с драйверами робота FootBot4.

Об интерфейсе CAN

Интерфейс определяет физический и канальный уровень модели OSI. Релевантной к задаче является спецификация канального уровня, которая будет приведена ниже.

Канальный уровень

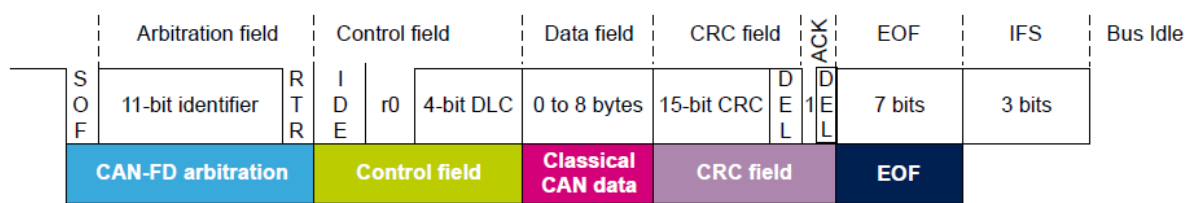
<https://habr.com/ru/articles/939978/>

Биты образуют массив, который формирует пакет CAN. Пакет содержит идентификаторы данные и служебные биты (подтверждение, длина поля данных). CAN пакет передает массив с цифрами (пакет). В пакете самое важное это 8 байт (64 бит) полезных данных. Это называют сообщением. Внутри сообщения есть параметры (они же сигналы). У каждого параметра есть номер. Параметры могут быть как вдавлены друг в друга, так и размазаны по битам поля данных. Максимум в одном сообщении может быть 64 параметра (сигнала) (по одному биту на параметр в предельном случае). В идеологии CAN классического запрос-ответ нет, как, например, в modbus. Обычно ноды просто непрерывно флудят в шину и тот, кому надо выхватывает то, что ему надо. Но это зависит от протокола прикладного уровня.

Главное преимущество интерфейса CAN - это разрешение коллизий налету, то есть без потери битовой скорости. Если два узла начнут передавать пакет, то продолжит передачу то устройство у которого меньше ID. Таким образом, ID определяет приоритет ноды во всей сети. Бинарная структура пакета CAN Classic пакета

Пакет - это по сути массив битов.

CAN 2.0: Classical base frame format



Бинарную структуру CAN пакета можно изучить тут: https://docs.google.com/spreadsheets/d/1-Lat8LepEZsLL8gNMegPnwGosyWMC_SJLOWZmXdXzMY/edit?gid=0#gid=0

1	11	1	1	1	4	8	8	8	8	8	8	8	8	15	1	1	7
SOF	ID	RTR remote transmission request	IDE	Ro	DLC	1	2	3	4	5	6	7	8	CRC	CRC delim	ACK	EOF
Начало кадра	Идентификатор	Запрос на передачу	Бит расширения идентификатора	reserv	Длина данных	data	data	data	data	data	data	data	data	CRC	Разграничитель контрольной суммы	Разгранич подтвержд	Конец кадра

Пояснения битовых полей представлены в этой таблице.

№	Битовое поле	Расшифровка	bits	Пояснение
1	SOF	Start of Frame	1	сообщающий о начале фрейма и позволяющий синхронизировать узлы после фазы ожидания;
2	ID	Identifier	11	Может иметь 11- или 29-битный размер. Устройство с меньшим ID выигрывает арбитраж. Соответственно, чем меньше ID, тем выше приоритет у устройства;
3	RTR	Remote Transmission Request	1	Если устройство передает данные, то бит RTR принимает рецессивное состояние. Если устройство запрашивает сообщение от другого узла – бит RTR принимает доминантное состояние;

4	IDE	бит-указатель на расширенный ID	1	Если IDE принимает доминантное состояние – это значит, что используется стандартный 11-битный идентификатор. Если IDE принимает рецессивное состояние, то принимающий контроллер должен быть готов к приему оставшейся части расширенного 29-битного идентификатора;
5	r0		1	резервный бит
6	DLC	Data Length Code	4	4-битное поле длины данных , которое кодирует, сколько байтов данных будет передано в сообщении. В классическом CAN DLC может принимать значение в диапазоне 0...8.
7	DATA	DATA	64	В классическом CAN фрейм может содержать 0...8 байтов данных
8	CRC	CRC	15	15-битный контрольный CRC-код для переданных данных
9	CRC D	Delimiter	1	бит-ограничитель для поля CRC
10	ACK		1	бит подтверждения сообщения. В исходном фрейме передатчик использует рецессивное состояние этого бита. В свою очередь приемники при отсутствии ошибок должны установить в этом бите доминантное состояние. Другими словами, если на шине есть хотя бы один активный приемник, то в этом бите будет установлено доминантное состояние;
11	ACK D		1	бит-ограничитель (Delimiter) для ACK
12	EOF	End-of-Frame	7	7-битное поле окончания фрейма
13	IFS	Interframe Space	any	меж фреймовый интервал, необходимый для того, чтобы приемник успел поместить сообщение в буфер.

Если трансивер передатчика не увидит бит подтверждения ACK, то передатчик (CAN MAC) будет снова и снова непрерывно передавать этот же CAN пакет, до тех пор пока его кто-нибудь не примет или случится bus-off. Таким образом, шина будет занята на 100 %. Однако некоторые микроконтроллеры всё же позволяют настроить MAC на одиночную отправку.

А это структура пакета с расширенным идентификатором (29 бит). Это позволяет адресовать больше узлов.

1	11	1	1	18	1	2	4	8	8	8	8	8	8	8	8	8	15	1	1	1	7
SOF	ID A	SRR	IDE	ID B	RTR	RES	DLC	1	2	3	4	5	6	7	8		CRC		ACK		EOF
Начало кадра	ID A	Подмена запроса на передачу	Бит расширения идентификатора	ID B	Запрос на передачу (RTR)		Len	data	data	data	data	data	data	data	data	data	CRC15	CRC delim	ACK	ACK delim	EOF

Еще бывает так делают, что берут Ext ID в 29 бит, адреса узлов ставят, условно, 22 бита. А самые старшие 3 выделяют под приоритет сообщения. В результате каждый ID имеет свои отдельные уровни приоритетов (8 уровней).

Table 15 — Example for source and destination address

29 bit CAN identifier																												
28	27	26	25	24	23	22	21							11	10						0							
Priority 0x6			ISO 15765 format		Type of service ISO 15765-3 messages		Source address 0x2ED							Destination address 0x32F														
1	1	0	1	1	1	1	0	1	0	1	1	1	0	1	1	0	1	0	1	1	0	0	1	0	1	1	1	1

Транспортный уровень

Для классического CAN, чтобы передавать пакеты превышающие 8 байт нужен транспортный протокол. Обычно таким транспортом является протокол ISO-TP. Он же ISO 15765-2.

Уровень приложения

В сетях на основе CAN на уровне приложения обычно гоняют следующие прикладные протоколы: CCP, XCP, CanOpen, UDS или J1939.

XCP позволяет гибко калибровать любой параметр и считывать любой сигнал, при этом в коде программы никаких действий для этого не нужно делать, т.к. XCP модуль напрямую общается с памятью, а вся информация о переменных хранится в файле с расширением *.a2l наподобие .elf.

UDS протокол тоже позволяет считать переменные и менять параметры, но эти параметры должны специальным образом быть пронумерованы прямо в прошивке ECU, потому что доступ к ним со стороны хоста идёт по ID а не по адресу в памяти. Также UDS поддерживает различные сервисы, например рутины - функции ECU, которые можно запустить с сервисного ПО. Прочитать и очистить ошибки, прочитать логи, перейти в загрузчик. Стандарт даёт огромную гибкость в реализации, и на разных ECU все сервисы и сам протокол может выглядеть очень по разному.

О протоколе ISO-TP

ISO 15765-2 или **ISO-TP** (Transport layer) - международный стандарт передачи пакетов данных по шине CAN. Протокол позволяет передавать пакеты размером более 8 байт, что превышает количество полезных байт данных, регламентируемых канальным уровнем интерфейса CAN.

Протокол ISO-TP работает на 3 (сетевом) и 4 (транспортном) уровне модели OSI.

Краткая спецификация протокола

https://en.wikipedia.org/wiki/ISO_15765-2

Протокол ISO-TP определяет 4 типа кадров:

Type	PCI Code	Description
Single frame (SF)	0	The single frame transferred contains the complete payload of up to 7 bytes (normal addressing) or 6 bytes (extended addressing)

Type	PCI Code	Description
First frame (FF)	1	First frame of a multi-frame packet, used when greater than 6/7 data bytes are to be communicated. The first frame contains the length of the full packet and the initial data.
Consecutive frame (CF)	2	A frame containing subsequent data for a multi-frame packet
Flow control frame (FC)	3	Response from the receiver, acknowledging a start of a multi-frame packet. Used to manage the pace of the consecutive frames.

Подробную спецификацию переписать сейчас не получилось, для ознакомления рекомендуется перейти по ссылке.

О протоколах уровня приложения

Протокол CanOpen

Протокол XCP

[https://en.wikipedia.org/wiki/XCP_\(protocol\)](https://en.wikipedia.org/wiki/XCP_(protocol))

<https://www.csselectronics.com/pages/ccp-xcp-on-can-bus-calibration-protocol>

Universal Measurement and Calibration Protocol (XCP) - протокол взаимодействия с электронными блоками управления (ECU - Electronic Control Unit), широко использующийся в автомобильной индустрии на ранних этапах разработки.

Позволяет мастеру на шине иметь доступ на чтение и запись к памяти блока управления. Структура памяти описывается файлами формата .a2l.

Протокол поддерживает чтение данных в режиме запрос-ответ (polling) и в режиме DAQ - Synchronous Data Acquisition.

Последний позволяет инициировать получение данных со слейва в виде единого синхронизированного потока значений, причем с возможностью тонкой настройки для каждого вида сигналов. Например, сигналы А и В можно запросить с периодом 10мс, сигнал Б с периодом 100мс или по определенному событию (например большой ток) и слейв будет посылать данные в указанном формате с указанной частотой.

Формат файлов .a2l

<https://www.csselectronics.com/pages/a2l-file-asap2-intro-xcp-on-can-bus>

Для описания внутренней структуры ECU используется формат файлов .a2l, краткое описание которого представлено по ссылке. Формат объемный и предоставляет большое количество возможностей для описания сигналов.

Резюме

Протокол предоставляет обширные возможности для взаимодействия с ECU, регламентируя чтение/запись памяти блока управления, инициализацию обратной связи для мониторинга и другие полезные функции. Однако поиск показал отсутствие библиотек и поддержки для реализации протокола на семействе микроконтроллеров STM32, что делает протокол неприменимым в нашей ситуации. Реализация его с нуля потребует неоправданно больших трудозатрат.

Протокол UDS

https://en.wikipedia.org/wiki/Unified_Diagnostic_Services

Unified Diagnostic Services (UDS) - это диагностический коммуникационный протокол, используемый в электронных блоках управления (ЭБУ) автомобильной электроники.

Протокол предоставляет возможность читать и писать в целевое устройство значения характеристик, вызывать процедуры и прочее, однако исходя из его спецификации не предполагает работу в реальном времени, что необходимо для общения с драйверами моторов.

Постановка задачи

Задачи материнской платы

В задачи материнской платы входит:

- Отправка команд управления по:
 - моменту
 - скорости
 - положению
- Получение обратной связи от драйверов:
 - ток (момент на валу)
 - скорость
 - положение
- Установка параметров драйвера:
 - коэффициенты регуляторов и фильтров
 - ограничения максимальных величин:
 - ток
 - скорость
- (Опционально) Прошивка драйвера по CAN

Задачи драйвера

- Обработка данного ему материнской платой задания
- Отправка телеметрии обратно на материнку

Технические ограничения

Характеристики МК материнской платы

МК: STM32F429ZIT6

Поддержка CAN-2.0B

Характеристики МК драйвера

МК: STM32G431

Поддержка FDCAN

Общие требования

Предполагаемая скорость передачи данных по шине CAN: 500кбит/с.

При полной загрузке пакетов полезной нагрузкой объем одного фрейма будет: 47 служебных бит + 64 бит полезной нагрузки = 111 бит на один фрейм.

Максимальная частота пакетов на шине:

$$\frac{1000\text{ kbit/s}}{111\text{ bit}} = 9009 \text{ Hz}$$

Определение частоты цикла системы управления

За один цикл материнская плата должна:

- Выдать задания по скорости/моменту на 4 маршевых привода
- Получить обратную связь от драйверов по интересующему параметру

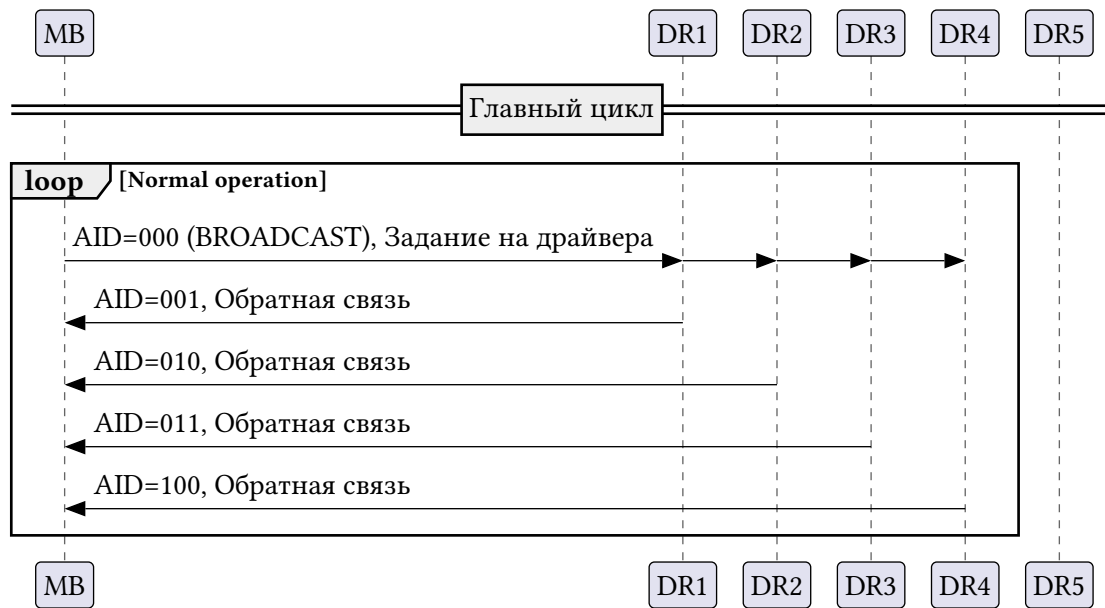


Рис. 1. Временная диаграмма связи материнской платы с драйверами

Таким образом за одну итерацию необходимо осуществить обмен 5 кадрами, что ограничивает максимальную частоту главного цикла 1801.8 Гц, или 555 мкс.

В качестве частоты главного цикла была выбрана частота 1000 Гц.

Спецификация протокола

Ввиду отсутствия готовых промышленных протоколов, подходящих под нашу задачу было принято решение разработать свой протокол передачи данных.

Ключевыми критериями при разработке протокола были:

- Компактность и максимальная простота реализации
- Минимизация количества служебных сообщений на шине

Для достижения этих целей было принято решение по максимуму использовать ID пакета для передачи служебной информации, такой как - идентификаторы получателя, тип посылки, требуемое действие (чтение, передача уставки, изменение режима работы).

Для запроса обратной связи, ради уменьшения количества пакетов на шине на драйверах выделен ряд регистров, задающих параметры требуемой обратной связи. Каждый раз после получения команды управления драйвер проверяет значения этих регистров и отправляет в ответе соответствующие данные.

Плюсы подхода

1. За счет инкапсуляции всех необходимых данных внутри ID не требуется дополнительных пакетов для запроса информации.
2. Передача данных, представляемых типами INT64 и FLOAT64, может быть осуществлена одним фреймом, что снижает накладные расходы на шину.

Формат заголовка

Устройств на шине 6:

- Материнская плата
- 5 драйверов моторов

Необходимости иметь отдельный идентификатор для материнской платы нет, поскольку она никогда не является целью для передачи данных.

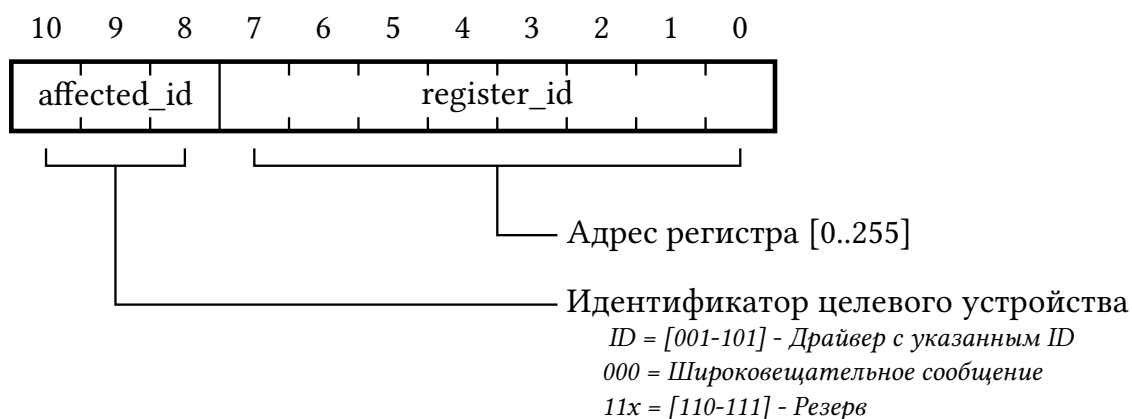


Рис. 2. 11-bit CAN identifier

• affected_id

Идентификатор целевого устройства - affected_id указывает на то, какое устройство должно получить/отправляет полезные данные.

В качестве целевого устройства может быть выбран либо конкретный драйвер (тогда его адрес кодируется в поле affected_id), либо все драйвера сразу (например при необходимости изменить режим работы драйвера с управления по току на управление по скорости, или при изменении коэффициентов и настроек звеньев насыщения).

Здесь стоит заметить, что на роботе 4 маршевых мотора и 1 мотор дрибблера. Таким образом драйвера разбиваются на две независимых группы, каждой из которых управлять необходимо по разному. Настройки для маршевых приводов неприменимы для мотора дрибблера и наоборот.

Данная проблема решается следующим образом: принимать широковещательные сообщения могут только драйвера с $ID \in \{1, 2, 3, 4\}$.

• register_id

Поле register_id хранит номер регистра с которым предполагается работа. Информация о регистрах представлена в разделе «Таблица регистров».

Полезная нагрузка

Размер полезной нагрузки переменный, определяемый типом данных в целевом регистре.

Тип и формат данных полезной нагрузки регламентируется форматом хранения данных внутри драйвера.

Таблица регистров

Данные внутри драйвера хранятся в регистрах переменного размера. Размер каждого регистра указан в таблице регистров.

При команде на запись происходит запись по адресу регистра в соответствии с количеством байт полезной нагрузки. В случае если полезная нагрузка по размеру больше, чем регистр -

может произойти переполнение буфера и изменится значение соседних регистров. Защиты от этого не предусмотрено.

При запросе на чтение содержимое соответствующего регистра посылается на шину в качестве полезной нагрузки.

#	Название поля	Описание
0	EMERGENCY_STOP [w] [ANY]	Аварийный останов. При записи любого значения драйвер полностью обесточивает электропривод на 15 секунд
1	SUPPLY_VOLTAGE [r] [FLOAT32]	Напряжение питания
4	MONITOR_SETTINGS [rw] [CANFuocoMonitorSettings]	Настройка мониторинга. Значение регистра указывает на то, какой регистр будет отправляться в обратной связи от драйвера struct CANFuocoMonitorSettings { CANFuocoRegisterMap register_to_monitor; };
-	-	-
	===== SimpleFOC =====	
	=== State variables ===	
10	TARGET [rw] [FLOAT32]	Текущая уставка. Смысл зависит от выбранного режима управления
11	FEED_FORWARD_VELOCITY [r] [FLOAT32]	Current feed forward velocity
12	SHAFT_ANGLE [r] [FLOAT32]	Current motor angle
13	ELECTRICAL_ANGLE [r] [FLOAT32]	Current electrical angle
14	SHAFT_VELOCITY [r] [FLOAT32]	Current motor velocity
15	CURRENT_SP [r] [FLOAT32]	Target current (q current)
16	SHAFT_VELOCITY_SP [r] [FLOAT32]	Current target velocity
17	SHAFT_ANGLE_SP [r] [FLOAT32]	Current target angle
18	VOLTAGE_D [r] [FLOAT32]	Current d voltage set to the motor
19	VOLTAGE_Q [r] [FLOAT32]	Current q voltage set to the motor
20	CURRENT_D [r] [FLOAT32]	Current d current measured
21	CURRENT_Q [r] [FLOAT32]	Current q current measured
22	VOLTAGE_BEMF [r] [FLOAT32]	Estimated backemf voltage (if provided KV constant)
23	U_ALPHA [r] [FLOAT32]	Phase voltages U alpha and U beta used for inverse Park and Clarke transform
24	U_BETA [r] [FLOAT32]	-//-
	=== Motor configuration parameters ===	
25	VOLTAGE_SENSOR_ALIGN [rw] [FLOAT32]	sensor and motor align voltage parameter

#	Название поля	Описание
26	VELOCITY_INDEX_SEARCH [rw] [FLOAT32]	target velocity for index search
	=== Motor physical parameters ===	
27	PHASE_RESISTANCE [rw] [FLOAT32]	Motor phase resistance
28	POLE_PAIRS [rw] [INT32]	Motor pole pairs number
29	KV_RATING [rw] [FLOAT32]	Motor KV rating
30	PHASE_INDUCTANCE [rw] [FLOAT32]	Motor phase inductance
	=== Limiting variables ===	
31	VOLTAGE_LIMIT [rw] [FLOAT32]	Voltage limiting variable
32	CURRENT_LIMIT [rw] [FLOAT32]	Current limiting variable
33	VELOCITY_LIMIT [rw] [FLOAT32]	Velocity limiting variable
	=== Motor status ===	
34	ENABLED [rw] [BOOL]	enabled or disabled motor flag
35	MOTOR_STATUS [r] [FOCMotorStatus]	Motor status <pre>enum FOCMotorStatus : uint8_t { motor_uninitialized = 0x00, //!< Motor is not yet initialized motor_initializing = 0x01, //!< Motor initialization is in progress motor_uncalibrated = 0x02, //!< Motor is initialized, but not calibrated (open loop possible) motor_calibrating = 0x03, //!< Motor calibration in progress motor_ready = 0x04, //!< Motor is initialized and calibrated (closed loop possible) motor_error = 0x08, //!< Motor is in error state (recoverable, e.g. overcurrent protection active) motor_calib_failed = 0x0E, //!< Motor calibration failed (possibly recoverable) motor_init_failed = 0x0F, //!< Motor initialization failed (not recoverable) };</pre>
	=== PWM modulation settings	
36	FOC_MODULATION [rw] [FOCModulationType]	Parameter determining modulation algorithm: SinePWM = 0x00, //!< Sinusoidal PWM modulation SpaceVectorPWM = 0x01, //!< Space vector modulation method Trapezoid_120 = 0x02, Trapezoid_150 = 0x03,

#	Название поля	Описание
37	MODULATION_CENTERED [rw] [BOOL]	Flag (1) centered modulation around driver limit /2 or (0) pulled to 0
	=== Control mode ===	
38	TORQUE_CONTROLLER [rw] [TorqueControlType]	enum TorqueControlType : uint8_t { voltage = 0x00, //< Torque control using voltage dc_current = 0x01, //< Torque control using DC current (one current magnitude) foc_current = 0x02, //< torque control using dq currents };
39	CONTROLLER [rw] [MotionControlType]	Режим управления мотором. При изменении обнуляет уставку enum MotionControlType : uint8_t { torque = 0x00, //< Torque control velocity = 0x01, //< Velocity motion control angle = 0x02, //< Position/angle motion control velocity_openloop = 0x03, angle_openloop = 0x04 };
-	-	-
	=== Controller and low pass filter settings ===	
40	PID_CURRENT_D_P [rw] [FLOAT32]	
41	PID_CURRENT_D_I [rw] [fFLOAT32]	
42	PID_CURRENT_D_D [rw] [fFLOAT32]	
43	PID_CURRENT_D_OUTPUT_RAMP [rw] [FLOAT32]	
44	PID_CURRENT_D_LIMIT [rw] [FLOAT32]	
45	LPF_CURRENT_D_TF [rw] [fFLOAT32]	
-	-	-
46	PID_CURRENT_Q_P [rw] [fFLOAT32]	
47	PID_CURRENT_Q_I [rw] [fFLOAT32]	
48	PID_CURRENT_Q_D [rw] [fFLOAT32]	
49	PID_CURRENT_Q_OUTPUT_RAMP [rw] [FLOAT32]	

#	Название поля	Описание
50	PID_CURRENT_Q_LIMIT [rw] [FLOAT32]	
51	LPF_CURRENT_Q_TF [rw] [fFLOAT32]	
-	-	-
52	PID_VELOCITY_P [rw] [fFLOAT32]	
53	PID_VELOCITY_I [rw] [fFLOAT32]	
54	PID_VELOCITY_D [rw] [fFLOAT32]	
55	PID_VELOCITY_OUTPUT_RAMP [rw] [FLOAT32]	
56	PID_VELOCITY_LIMIT [rw] [FLOAT32]	
57	LPF_VELOCITY_TF [rw] [fFLOAT32]	
-	-	-
58	P_ANGLE_P [rw] [fFLOAT32]	
59	P_ANGLE_LIMIT [rw] [FLOAT32]	
60	LPF_ANGLE_TF [rw] [fFLOAT32]	
-		
61	MOTION_DOWNSAMPLE [rw] [UINT32]	
62	MOTION_CNT [rw] [UINT32]	
-		
	=== Sensor settings ===	
63	SENSOR_OFFSET [rw] [FLOAT32]	
64	ZERO_ELECTRIC_ANGLE [rw] [FLOAT32]	
65	SENSOR_DIRECTION [rw] [Direction]	
66	PP_CHECK_RESULT [rw] [BOOL]	
-	-	-
	===== Virtual registers =====	
96	MULTI_TARGET [w] [FLOAT16]x4	Текущая уставка. Используется в комбинации с широковещательными пакетами, чтобы выставить уставку на 4 мотора сразу. Содержит 4 числа FLOAT16. Актуальное значение уставки определяется исходя из ID мотора
102	SHAFT_FEEDBACK [r] [FLOAT32]x2	Значения SHAFT_VELOCITY и SHAFT_ANGLE
127	SAVE_TO_EEPROM [w] [VOID]	При записи любого значения сохраняет текущие параметры настройки мотора в энергонезависимую память, откуда они будут восстановлены при следующем перезапуске NOT IMPLEMENTED

Описание виртуальных регистров

Регистр MULTI_TARGET

Данные в регистре хранятся в формате FP16 (1-5-10).

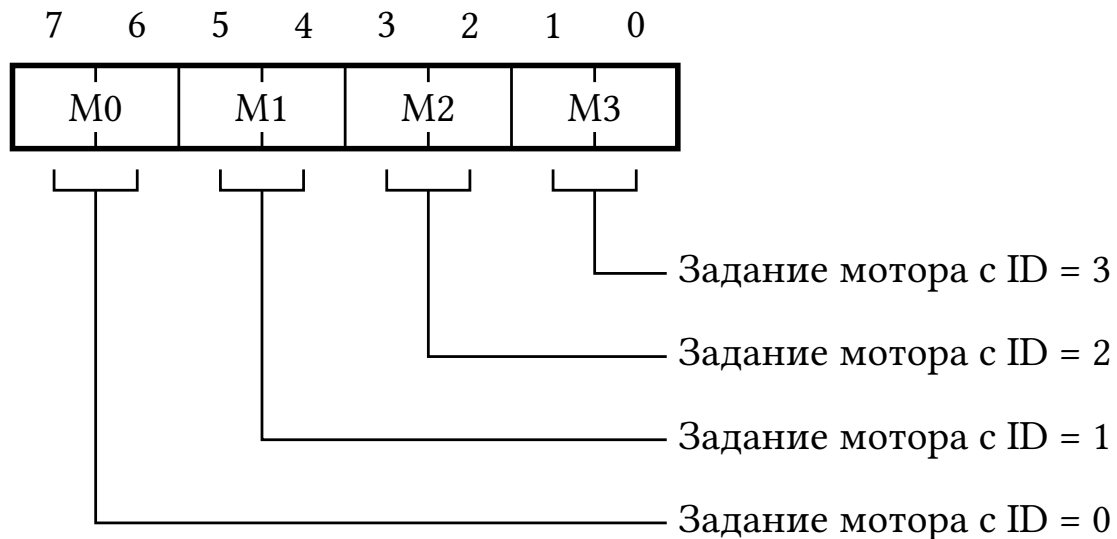


Рис. 3. Структура регистра MULTI_TARGET

Регистр SHAFT_FEEDBACK

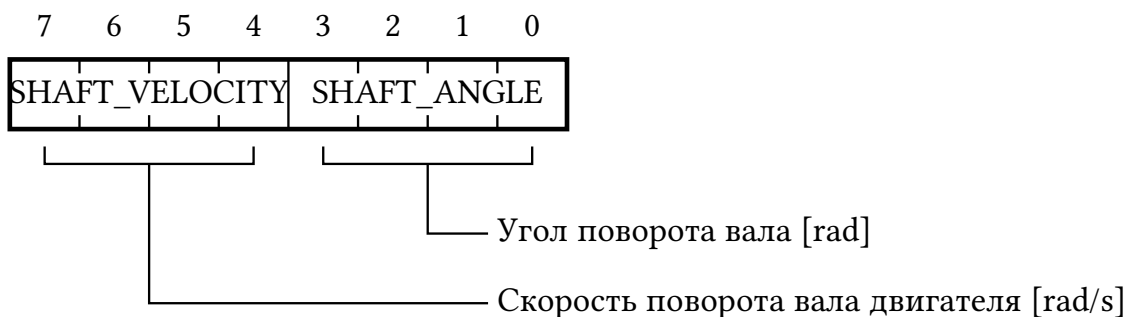


Рис. 4. Структура регистра SHAFT_FEEDBACK

Спецификация FP16

Стандарт IEEE 754 [1] определяет формат `binary16` как изображено на Рис. 5. Использование 10 бит под мантиссу обеспечивает поддерживаемую точность $\log_{10}(2^{10+1}) \approx 3.311$ значащих цифр, чего достаточно для целей управления двигателями всенаправленного робота.

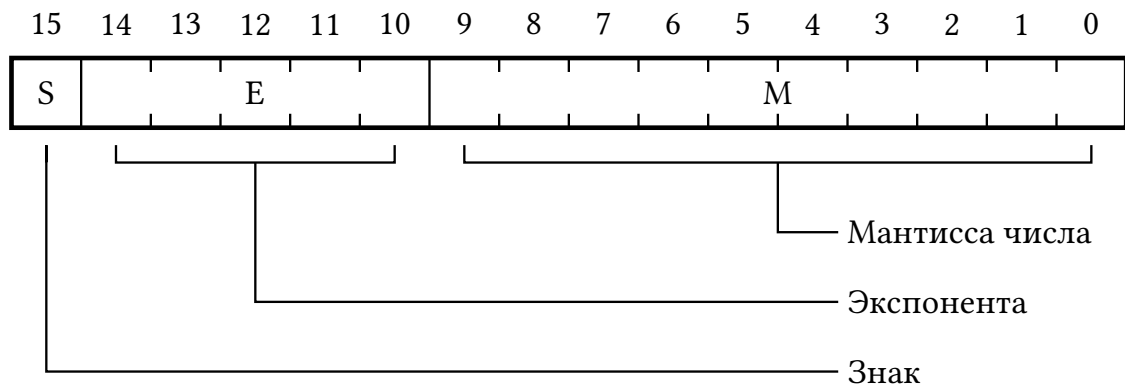


Рис. 5. Структура данных в формате FP16

При передаче уставки на робота нет необходимости кодировать числа NaN и $\pm \text{inf}$, что позволяет увеличить диапазон возможных чисел и упростить алгоритм пересчета. Сами алгоритмы взяты из [2]. Их реализация приведена в Листинг 1.


```

1  #pragma once
2
3  // https://stackoverflow.com/a/60047308
4
5  typedef unsigned short ushort;
6  typedef unsigned int uint;
7
8  uint as_uint(const float x)
9  {
10     return *(uint *)&x;
11 }
12 float as_float(const uint x)
13 {
14     return *(float *)&x;
15 }
16
17 float half_to_float(const ushort x)
18 {
19     const uint e = (x & 0x7C00) >> 10;
20     const uint m = (x & 0x03FF) << 13;
21     const uint v = as_uint((float)m) >> 23;
22     return as_float((x & 0x8000) << 16 | (e != 0) * ((e + 112) << 23 | m) | ((e
    == 0) & (m != 0)) * ((v - 37) << 23 | ((m << (150 - v)) & 0x007FE000)));
23 }
24 ushort float_to_half(const float x)
25 {
26     const uint b = as_uint(x) + 0x00001000;
27     const uint e = (b & 0x7F800000) >> 23;
28     const uint m = b & 0x007FFFFF;
29     return (b & 0x80000000) >> 16 | (e > 112) * (((e - 112) << 10) & 0x7C00) |
    m >> 13 | ((e < 113) & (e > 101)) * (((0x007FF000 + m) >> (125 - e)) + 1)
    >> 1 | (e > 143) * 0x7FFF;
30 }

```

Листинг 1. Функции перевода между FLOAT32 и FP16

Библиография

- [1] «IEEE Standard for Floating-Point Arithmetic», *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, cc. 1–84, июл. 2019, doi: 10.1109/IEEESTD.2019.8766229.
- [2] M. Lehmann, M. J. Krause, G. Amati, M. Sega, J. Harting, и S. Gekle, «Accuracy and performance of the lattice Boltzmann method with 64-bit, 32-bit, and customized 16-bit number formats», *Physical Review E*, т. 106, вып. 1, с. 15308, июл. 2022, doi: 10.1103/PhysRevE.106.015308.